# Target Specification, Not Model Capability, Explains Most Unit-Test Generation Failures at Medium Complexity

Nguyễn Tùng Ân, Đoàn Thị Thu Kim, Sơn Hải, Trương Hoàng Phúc, and Trần Xuân Lộc

Group 2 — SE1944, FPT University, Vietnam
Supervisor: L.T.Q.Chi

**Abstract.** Failures of LLM-generated unit tests are usually recorded against the model. Examining them individually, we find most belong to the benchmark and the measurements instead. Sampling by cyclomatic complexity alone admits targets no external test can reach: of 1,848 functions in the band $5 \leq CC \leq 10$ across ten projects, only 474 — 26 % — are simultaneously public and unambiguously named. We rebuild the benchmark under five testability criteria fixed in advance, sample 60 Java and 60 Python functions ($n = 59$ for Java after excluding one missampled target), and score every suite at four nested tiers, resting all claims on mutation score inside the target's line range — the only tier a suite that verifies nothing cannot satisfy. Two one-shot prompts differing in exactly one variable isolate the effect of supplying the target specification. On Python it raises the proportion of suites reaching the target from 29/60 to 42/60 ($p = 0.003$, rank-biserial +0.61), but the proportion detecting a fault only from 16/60 to 22/60 ($p = 0.33$): the prompt improves arrival, not discrimination — a split branch coverage alone would have concealed. Against established generators the ordering reverses by language: the model beats Pynguin decisively on Python, where Pynguin detected faults on 2 of 60 targets under either configuration ($p \leq 0.003$ in every pairing), while on Java both EvoSuite and Randoop detect faults on more targets, significantly so only for EvoSuite under the stricter gate ($p = 0.028$). Six measurement defects surfaced, four introduced while repairing the other two; none produced an error message.

**Keywords:** Unit test generation · Large language models · Mutation testing · Cyclomatic complexity · Measurement validity

## 1 Introduction

A large language model asked to write a unit test for a specific function will often produce something that does not compile. It is tempting to read that as a limit of the model. In an earlier study of `gpt-4o-mini` on 120 medium-complexity functions we read it that way ourselves. This paper reports what we found when we examined the failures individually instead, and the answer turned out to be mostly about the benchmark and the measurements rather than about the model.

Two problems surfaced, and they compound.

*The benchmark contains targets nothing can test.* Sampling functions by cyclomatic complexity is standard, and it says nothing about whether a sampled function is reachable from an external test. In the ten projects studied here, 1,848 functions fall in the band $5 \leq CC \leq 10$; only 474 of them — 26 % — are simultaneously public and unambiguously named. A generator that fails on the other 74 % has been asked to do something no tool can do, and the resulting failure is recorded as a failure of the generator. Attributing a method to the wrong declaring class has the same effect and is easy to do silently: a brace counter that does not pop on block exit assigns a method to whichever nested type was declared most recently.

*The usual metrics can be satisfied without testing anything.* Compilation success and coverage are the two most commonly reported outcomes in this literature. Both can be obtained by a suite that verifies nothing. Reflection turns every identifier into a string, so a call naming a method that does not exist compiles with return code 0; and a generated suite in our own corpus consists of a single assertion-free call to the wrong function, which passes. The consequence is not that these metrics are noisy — it is that a number can be reported when the underlying measurement has stopped working, with nothing to signal it. We encountered this six times during the study, and four of those were defects we introduced ourselves while repairing the other two (Section 6).

### This paper

We rebuild the benchmark under five testability criteria fixed in advance, sample 60 Java and 60 Python functions from the surviving pool, and measure every suite at four nested tiers, resting all claims on mutation score inside the target's line range — the only tier that cannot be satisfied vacuously. Two *one-shot* prompt conditions differing in exactly one variable isolate the effect of supplying the target specification, and three established generators (EvoSuite, Randoop, Pynguin) provide reference points. The sampling criteria, tool versions, budgets and hypotheses were committed to a public repository before any measurement was run.

Three findings stand out.

1. **Target specification improves reach, not fault detection.** On Python, adding the fully qualified target, its signature, and an executed constructor call raises the proportion of suites that reach the target from 29/60 to 42/60 ($p = 0.003$, rank-biserial +0.61). The proportion that detects a fault rises from 16/60 to 22/60, which is *not* significant ($p = 0.33$). The prompt gets the test to the right place; it does not make the test check anything. A study

reporting only coverage would have called this an unambiguous improvement.
2. **The LLM and the search-based tools win in different languages.** On Python, both prompt conditions beat Pynguin decisively — 2/60 for Pynguin against 16/60 and 22/60 — at $p \leq 0.003$ in every pairing, and this holds under both of the two Pynguin configurations we ran. On Java the count ordering reverses, with both mature search-based tools detecting faults on more targets than the model (significantly so for EvoSuite under the stricter gate). Neither result generalises to "LLMs are better" or "LLMs are worse."
3. **Most of the attrition is structural.** Across both languages the dominant obstacle is not writing assertions but obtaining an instance to assert about. This is the same barrier the search-based literature identified for object-oriented code, and the model does not escape it.

We report the sampling funnel in full, both Pynguin configurations side by side, and both settings of a green-test gate whose Java implementation we found to disagree with its own pre-registered definition. Where a repair moved results in our own favour we say so explicitly and give the numbers under both settings, so that no conclusion in this paper depends on which of the two a reader prefers.

## 2 Related Work

### 2.1 Search-Based and Random Test Generation

Automated unit-test generation predates language models by more than a decade. Randoop builds test sequences incrementally and uses feedback from executing each prefix to discard sequences that are illegal or redundant [11]. EvoSuite treats an entire test suite as one individual in a genetic search and evolves it against a coverage-based fitness function [3]. Both have been evaluated at scale: EvoSuite reached high branch coverage across a large sample of open-source classes, while also showing that a substantial fraction of classes remain hard for reasons unrelated to the search itself [4]. Pynguin ports the search-based approach to Python, where the absence of static types removes much of the information the Java generators depend on [8,9].

Two findings from this line of work bear directly on our design. First, coverage and fault detection are not interchangeable: Shamshiri et al. found that automatically generated suites with high coverage detected only a minority of real faults [14], and Inozemtseva and Holmes report that coverage correlates with suite effectiveness far more weakly than is commonly assumed [5]. Second, the difficulty of a target is not captured by its complexity alone — constructing the receiver object is frequently the binding constraint. We return to both points in Sections 3.3 and 5.

### 2.2 LLM-Based Test Generation

Code-trained language models made test generation a prompting problem [2], and the earliest work in this direction fine-tuned transformers on method–test pairs with surrounding focal context [16]. Subsequent systems moved to prompting. TestPilot generates tests for JavaScript functions from documentation and usage examples, and re-prompts with the failure message when a test does not pass [13]. ChatTester adds an iterative repair loop driven by compiler and runtime errors, reporting large gains over a single-shot baseline [18]. Siddiq et al. evaluate several models on JUnit generation and find compilation failure — not weak assertions — to be the dominant failure mode [15]. A recent survey catalogues the space and notes that evaluation practice is uneven across the field [17].

Our study is deliberately narrower than these systems in one respect and stricter in another. We use a *single* API call per function, with no repair loop and no retrieval, because a repair loop conflates the quality of the initial generation with the quality of the error signal fed back into it; when compilation failure is itself the object of study, that conflation is fatal. Where those systems ask how high the success rate can be driven, we ask what the initial generation is actually missing, and whether it is something a prompt can supply. The two prompt conditions of Section 3.2 differ in exactly one variable so that the answer is attributable to that variable.

### 2.3 Evaluation Metrics and Their Validity

Mutation analysis is the standard fault-based criterion for test-suite adequacy, and mutants have been shown to be a reasonable — if imperfect — substitute for real faults in controlled comparisons [6,12]. It is comparatively expensive, which is why much of the LLM test-generation literature reports compilation success and line or branch coverage instead.

This substitution is where our contribution sits. Compilation success and coverage are cheap because they ask less, and what they ask can be satisfied without testing anything: a reflective call compiles regardless of whether the named method exists, and a suite that constructs no object and calls nothing still passes. Neither weakness is hypothetical; both occur in the corpus measured here (Section 3.3). We therefore report four nested tiers and rest every claim on the only tier that cannot be satisfied vacuously.

A second and less discussed threat concerns the benchmark rather than the metric. Sampling targets by cyclomatic complexity [10] is standard practice, but complexity does not imply reachability: a method may be private, may be one of several overloads sharing a name, or may belong to a class that cannot be instantiated from outside. A suite that fails on such a target tells us about the sampling, not about the generator. To our knowledge, the fraction of a complexity-sampled benchmark that is actually addressable from an external test is rarely reported. We report the full sampling funnel for this reason, and the attrition turns out to be the study's most transferable number.

### 2.4 Methodological Practice

Because the present work re-examines a dataset after having seen results obtained on an earlier one, the distinction between repairing an instrument and selecting a favourable outcome is central. Kerr's account of hypothesising after the results are known [7] describes the failure mode precisely. We address it in two ways: the sampling criteria, tool versions, budgets and hypotheses were committed to the repository before any measurement was run, and the earlier study's dataset and reported numbers are left unchanged — this paper stands beside it rather than replacing it. For the statistical comparisons we follow the guidance of Arcuri and Briand on non-parametric testing and effect sizes for randomised algorithms in software engineering [1].

## 3 Methodology

### 3.1 Dataset Construction

We mine functions from ten open-source repositories pinned at fixed commits (eight Java projects: `commons-cli`, `commons-csv`, `commons-collections`, `commons-math`, `gson`, `jsoup`, `joda-time`, `jfreechart`; two Python projects: `flask`, `requests`). Cyclomatic complexity is measured with Lizard 1.23.0 and we keep the medium band $5 \leq CC \leq 10$, matching the complexity range studied in our earlier work.

Complexity alone, however, does not make a function a valid test target. A study that samples on complexity but ignores *testability* measures two things at once: how well a generator writes tests, and how often the sampled target could have been tested at all. We therefore apply five additional criteria, each fixed before any test was generated and each justified by whether *any* tool — an LLM, a search-based generator, or a human — could reach the target through the public API.

**F1** — **Non-trivial body.** $nloc \geq 3$. Removes stubs, abstract declarations and interface signatures, which contain no branches to cover.

**F2** — **Public or exported.** Java: `public` only. Python: names not prefixed with an underscore. A `private` method cannot be invoked from an external test class by ordinary means; reflection can reach it, but then the compiler no longer checks any name in the test (Section 3.3), so compilation ceases to be evidence of anything.

**F3** — **Unambiguous name.** Java: the method name occurs exactly once in its declaring class. Python: the function name occurs exactly once in its module. An overloaded name cannot be specified unambiguously in a prompt, and the compiler rejects the resulting call as ambiguous.

**F4** — **Verified declaring class.** The enclosing class is resolved with a brace-depth stack that *pops* on block exit. Taking "the nearest enclosing class declaration" without popping attributes a method to whichever nested class was declared most recently, which is wrong whenever the target follows a nested type in the same file.

**Table 1.** Sampling funnel. Each row reports how many candidates survive the criterion. F4 corrects the declaring-class attribution that F3 depends on rather than removing candidates in its own right, so it has no separate row.

| Stage | Criterion | Remaining |
|---|---|---|
| F0 | $5 \leq CC \leq 10$ (Lizard) | 1,848 |
| F1 | non-trivial body | 1,848 |
| F3 | unambiguous name | 753 |
| F2 | public / exported | 474 |
| F5 | dynamically resolvable (Python) | 64 |
| | final pool | Java 406, Python 64 |
| | sampled | Java 60, Python 60 |

**F5 — Dynamically resolvable** (Python). The module is imported, the qualified name is walked with `getattr`, and the result is confirmed callable. Syntactic inference is not evidence that a target can be reached at run time.

Table 1 reports the full funnel. We report every count-reducing stage rather than only the final count, because the attrition is itself a finding: of 1,848 functions in the complexity band, only 474 (26 %) are simultaneously public and unambiguous. Roughly three quarters of medium-complexity functions in these projects are not valid targets for an external test generator.

From the surviving pool we sample 60 Java and 60 Python functions, balanced across repositories (Java: 3–9 functions per project; Python: 32 from `requests`, 28 from `flask`). Complexity is concentrated at the lower end of the band ($CC$=5: 56 functions; $CC \geq 8$: 23), and the median function is 17 lines. Of the Python targets, 34 are instance methods and 26 are module-level functions. Every one of the 120 targets was confirmed resolvable at run time before generation began. One Java target was later found to violate F2 — our own parser had recorded a `protected` method as public — and is excluded, so Java results are reported over $n = 59$ and Python over $n = 60$ (Section 6).

### 3.2 Test Generation

All tests are produced by `gpt-4o-mini-2024-07-18` at `temperature` = 0, `top_p` = 1.0, `max_tokens` = 2048, in a **single API call per function** with no feedback loop and no retrieval.

We compare two *one-shot* prompt conditions. Both contain exactly one worked exemplar (a trivial `add` function with its test), so both are one-shot in the standard sense, and both issue one call. They differ in exactly one respect:

**V1 — baseline.** The prompt names the target function and shows its source. This is the prompt used in our earlier study.

**V2 — target-specified.** Identical to V1, with one additional block inserted before the task: the fully qualified target name, its exact signature, its declaring class, and — for instance methods — a constructor call that has been *executed* and confirmed to build the receiver successfully.

Holding the exemplar fixed matters. An earlier draft of the enriched prompt was rewritten from scratch and, in the process, dropped the exemplar; it therefore differed from V1 in two respects at once (no exemplar, plus target metadata) and no causal attribution was possible from it. We report that variant only as a secondary observation and label it zero-shot, since it is one.

### 3.3 Measurement: Four Tiers

A single "valid / invalid" flag conflates outcomes that differ in kind, so we report four nested tiers. Each answers a different question, and — importantly — the first two can be satisfied by a test that verifies nothing.

**T1 — Compiles / collects.** The test file is syntactically and type-correct.
**T2 — At least one green test.** At least one test passes against the unmodified source.
**T3 — Reaches the target.** Branch coverage $> 0$ within the target's line range $[start, end]$.
**T4 — Detects faults.** Mutation score $> 0$ within the same line range.

Only T4 supports the claim that a suite *tests* the target. We show that T1 and T2 are both reachable without testing anything:

- **T1 under reflection.** Reflection turns every name into a string, so the compiler stops checking it. A call to `getDeclaredMethod("aNameThatDoesNotExist")` compiles with return code 0 and fails only at run time. Any compilation-based validity rate is therefore uninformative for suites that use reflection.
- **T2 under an assertion-free suite.** The search-based suite generated for `requests.hooks` (target `dispatch_hook`) contains exactly one test, and that test is a single bare call to a *different* function in the same module, with no assertion of any kind. It compiles, it collects, and it passes — T1 and T2 are both satisfied — while measured branch coverage inside the target's line range is 0. A green-test criterion records this as a success.

The two tiers are therefore not merely weaker than T4; they can be satisfied by suites that are, respectively, unchecked by the compiler and empty of assertions.

Branch coverage is measured with `coverage.py` (Python), attributed to the target's line range rather than to the whole file; T3 is therefore reported for Python only. The Java pipeline executes the generated JUnit suites through the JUnit Platform launcher and rests on T4, so no Java branch-coverage figure is claimed anywhere in this paper. Mutation analysis applies the same operator set to both languages — arithmetic ($+/-$, $\times/\div$), relational ($>$ / $\geq$, $<$ / $\leq$), equality ($==/\neq$), boolean ($\wedge/\vee$), boolean constant inversion, and numeric constant increment — restricted to the target's line range and capped per function by language (20 mutants for every Java suite, 30 for every Python suite). The cap binds two Java functions and no Python one (Section 6); it does not affect T4, which asks only whether any mutant is killed.

Two procedural safeguards are worth stating because their absence silently corrupts results. First, mutation requires overwriting the source; we mutate a *copy* of the source tree placed ahead of the original on the import path, so that concurrent measurements reading the same tree are unaffected. Second, tool versions must be matched across families: JUnit 5 pairs Jupiter 5.x with Platform 1.x, and mixing a Jupiter 6 artifact with JUnit 5-style tests makes the launcher discover zero tests while exiting successfully — a failure that reports as a legitimate zero.

### 3.4 Baselines

Each baseline generates per class (Java) or per module (Python), matching how the tools operate; results are then attributed to individual functions by line range, exactly as for the LLM suites.

**EvoSuite 1.2.0** , 60 s per class, run under JDK 11. EvoSuite performs deep reflection into JDK internals, which the module system blocks from Java 9 onward, and its master process launches its client from `JAVA_HOME` — so both must point at the same JDK, or the client dies while the master still exits successfully. Because the projects were compiled with JDK 17, we recompiled sources to Java 11 bytecode into a separate output directory, leaving the original build untouched.

**Randoop 4.3.3** , 60 s per class, JDK 17.

**Pynguin 0.45.0** , 90 s per module, under **two configurations reported side by side**. In its default configuration Pynguin serialises module state to a worker process with `dill` and aborts on C-extension type objects that cannot be resolved by name, producing no output at all for several modules. We therefore also run it with the master–worker split disabled. Reporting both separates “the tool scored low” from “the tool never ran”.

### 3.5 Statistical Analysis

Conditions are compared with the paired Wilcoxon signed-rank test, matching on function identifier, at $\alpha = 0.05$, with matched-pairs rank-biserial correlation as effect size. Java and Python are analysed separately throughout: the toolchains differ, and Java enforces access control while Python does not, so their failure modes are not commensurable. Every rate is reported over the functions of its own language ($n = 59$ for Java after the exclusion above, $n = 60$ for Python).

Results are given at two levels in parallel: *all* ($n = 59$ for Java and $n = 60$ for Python, with unmeasurable cases scored 0) and *effective* (the subset with a non-zero value). The *all* level is reported even where the *effective* level is more favourable.

**Table 2.** Fault-detection rate (T4), original versus rebuilt dataset, same prompt (V1).

| Language | Green gate | Original | Rebuilt |
|---|---|---|---|
| Python | per-test | 13/60 (21.7 %) | 16/60 (26.7 %) |
| Java | whole-suite | 6/60 (10.0 %) | 12/59 (20.3 %) |

All configuration decisions above — criteria, tool versions, budgets, the two-branch Pynguin design, and the hypotheses — were recorded in a pre-registration committed before any baseline was executed.

## 4 Results

We report Python and Java separately throughout. Every rate is over the functions of its own language: $n = 60$ for Python and $n = 59$ for Java, the latter after excluding one mis-sampled target (Section 6). The primary tier is T4, fault detection within the target's line range; T3, reaching the target, is reported alongside it because the two diverge.

### 4.1 RQ-A: Clean Versus Original Dataset

Table 2 contrasts the fault-detection rate of the baseline prompt on the original benchmark with its rate on the rebuilt one. This is not a controlled comparison — the two are different sets of functions, and the rebuilt one is easier by construction — so we report only the size of the gap, and only between measurements taken with the *same* green-test gate, since a gate mismatch alone would inflate the difference.

The rebuilt dataset roughly doubles the Java rate and lifts the Python rate by about a quarter (13/60 to 16/60). Because the two samples are not matched, we draw no causal conclusion; the point is only that a large part of the failure rate reported on a complexity-only benchmark is attributable to targets that were never addressable, and removing them changes the headline number substantially.

### 4.2 RQ-B: Effect of Target Specification

Both prompt conditions are one-shot and differ in a single block of target metadata (Section 3.2). Table 3 reports both tiers for Python, where the result is decisive and, more importantly, splits between the two tiers.

The target specification raises the proportion of suites that reach the target from 29/60 to 42/60, a significant effect ($p = 0.003$). It raises the proportion that detects an injected fault from 16/60 to 22/60, which is not significant ($p = 0.33$). On the non-zero subsets the median scores barely move — T3 from 75.0 to 79.2, T4 from 84.4 to 88.9 — so the effect is almost entirely in *how many* suites clear each bar, not in how well the clearing suites do. A study reporting coverage

**Table 3.** Python, V1 versus V2. Paired Wilcoxon signed-rank, $\alpha = 0.05$; $r_{rb}$ is matched-pairs rank-biserial.

| Tier | V1 | V2 | $p$ | $r_{rb}$ | |
|---|---|---|---|---|---|
| T3 (reaches target) | 29/60 | 42/60 | 0.0027 | +0.61 | significant |
| T4 (detects fault) | 16/60 | 22/60 | 0.3259 | +0.28 | n.s. |

**Table 4.** Fault detection (T4) of the model versus baselines. Java $n = 59$, Python $n = 60$. GPT columns are the target-specified prompt (V2); the paired test compares that column with each baseline.

| Language Setting | GPT (V2) | Baseline | Paired test ($p$, $r_{rb}$) |
|---|---|---|---|
| *Java — EvoSuite* | | | |
| per-test | 16/59 | 23/59 | 0.14, +0.32 (n.s.) |
| whole-suite | 10/59 | 24/59 | 0.028, +0.51 (sig.) |
| *Java — Randoop* | | | |
| per-test | 16/59 | 19/59 | 0.86, −0.04 (n.s.) |
| whole-suite | 10/59 | 18/59 | 0.49, +0.18 (n.s.) |
| *Python — Pynguin* | | | |
| default | 22/60 | 2/60 | 0.0001, −0.96 (sig.) |
| no split | 22/60 | 2/60 | 0.0001, −0.96 (sig.) |

alone would read this as a clear improvement; the fault-based tier shows the improvement does not reach the point of testing behaviour.

The pre-registered hypothesis H-B predicted that the enriched prompt would not outperform the baseline. It holds at T4 and fails at T3. We record it as partially refuted, since the significant T3 effect is in the direction H-B predicted against.

On Java the same intervention changes almost nothing. Under the per-test gate, both prompts detect faults on 16 of 59 targets; under the whole-suite gate the counts are 12 (V1) and 10 (V2). The compile-failure count is identical — 38 functions under each prompt, 35 of them the same — so the target block, which resolves identity, does not clear the type-level barrier that dominates Java compilation. Only 4 functions differ in a way the paired test could use, which is below the six-pair minimum for a Wilcoxon test; we report the counts and the mechanism (Section 5.1) rather than a $p$-value.

### 4.3 RQ-C: Against Established Generators

Table 4 places the model beside the search-based and random generators. Java is reported under both green-test gates; Python under both Pynguin configurations. No conclusion below depends on which setting a reader adopts.

The two languages disagree, and the disagreement is the result. On Python the model dominates Pynguin under every pairing: against either configuration, and using either prompt, the paired test rejects at $p \leq 0.003$ with rank-biserial correlations between −0.81 and −0.96. The gap is driven by coverage of the benchmark rather than by score: Pynguin produced no suite at all for 39 of 60 targets in its default configuration, and disabling the master–worker split — the documented workaround for the serialisation crash responsible — reduced that to 35 while leaving T4 unchanged at 2/60. That net movement of four conceals considerable churn: 17 targets gained a suite and 13 lost one, and 14 of the 17 newly generated suites failed at collection.

On Java the ordering reverses in the counts. Both baselines detect faults on more targets than the model in every pairing — EvoSuite 23 or 24 versus 16 or 10, Randoop 19 or 18 versus 16 or 10 — but the paired Wilcoxon test reaches significance in only one of the four comparisons: EvoSuite under the whole-suite gate ($p = 0.028$, $r_{rb} = +0.51$). The other three, including both Randoop comparisons, do not reject at $\alpha = 0.05$ ($p$ from 0.14 to 0.86). So the Java evidence is a consistent count advantage for the search-based tools that is statistically firm only for EvoSuite once the stricter gate is applied; it is not the decisive reversal the raw counts might suggest. Even so, it points the opposite way from Python, where the model rejects against Pynguin at $p \leq 0.003$ in every pairing. Java is the setting these tools were built for, and where static types give a search-based generator the call information the model was shown to lack (Section 5.1). We therefore report no single verdict on whether the model beats automated generators: the count ordering is opposite in the two languages, and a study confined to either would have reached the other's headline.

## 5 Discussion

### 5.1 Three Barriers, Only the First of Which a Prompt Can Remove

The two prompt conditions differ in one variable, and the effect of that variable separates cleanly by language. On Python, supplying the target specification cut the number of suites that fail to import or collect from 17 to 4, rescuing 14 targets and breaking 1. On Java it changed effectively nothing: 38 suites failed to compile under the baseline prompt and 38 failed under the target-specified prompt, with 35 of the 38 being the same functions. Three functions moved in each direction, which is noise, not an effect.

The asymmetry is not a quirk of the two languages' tooling. It follows from what the added block contains. The block supplies *identity*: the fully qualified name, the declaring class or module, the signature, and a constructor call for the receiver. Python resolves identity at run time, and at the stage where these suites were failing, identity was the entire barrier — importing a module under the wrong path, calling a function that does not exist, invoking a method on the wrong class. Supply the identity and the barrier disappears. Java resolves identity *and types* at compile time. Having the right name still leaves the model to synthesise type-correct calls into the surrounding API: the right constructor arity and parameter types, the right return types, declared exceptions, generics. The target block does not touch any of that.

This suggests a three-level account of where generated tests fail, in order:

1. **Addressing** — calling the right thing at all. A target specification removes this barrier, and in Python it removes almost all of it.
2. **Typing** — calling it in a way the compiler accepts. Java only. The target specification does not help, because the difficulty lies in the API surrounding the target rather than in the target itself.
3. **State** — constructing a receiver in a configuration under which the behaviour of interest is observable. Neither prompt condition addresses this.

The evidence for the third level being untouched is direct. In Python, the count of suites that import successfully but have no passing test barely moved: 13 under the baseline prompt, 14 under the target-specified one, with only 7 functions common to both sets. Removing the addressing barrier did not shrink the state barrier; it *exposed* it. The population changed while the size did not.

That the binding constraint is receiver construction rather than assertion writing is not a new observation — it is what the search-based literature reports for object-oriented code [4,14]. The finding here is that a language model, which is not searching at all, runs into the same wall at the same place, and that handing it the constructor is insufficient: our constructor snippets were executed and verified to build the receiver, and the state barrier persisted anyway. Building *an* instance is not the same problem as building an instance in the state that makes a branch reachable.

### 5.2 Reaching a Target and Testing It Are Different Achievements

The clearest result in this study is a split between two tiers that a single-metric study would have merged. On Python, the target specification raised the number of suites reaching the target from 29/60 to 42/60, significant at $p = 0.003$ with a rank-biserial correlation of +0.61. It raised the number detecting an injected fault from 16/60 to 22/60, which is not significant ($p = 0.33$).

A study reporting branch coverage alone would have concluded that the enriched prompt is a clear improvement, and would have been wrong about what improved. The suites arrive at the right function more often; they do not check its behaviour more often. Coverage measures arrival. Mutation score measures discrimination, and only the latter supports the claim that a suite tests anything. That coverage overstates suite effectiveness is a known result [5]; what this experiment adds is that a specific, plausible intervention moves the two metrics by different amounts and in a way that inverts the conclusion drawn from them.

This is also the pre-registered hypothesis H-B coming apart. We predicted, before seeing any number, that the enriched prompt would *not* outperform the baseline. That prediction holds at T4 and fails at T3. We report it as partially refuted rather than reinterpreting it, since the direction of the T3 result is the one we predicted against.

### 5.3 Neither Family Wins Outright

The comparison against established generators does not resolve in a single direction, and the two languages disagree.

On Python, both prompt conditions beat Pynguin by a wide margin — 16/60 and 22/60 against 2/60 — at $p \leq 0.003$ in every pairing, with rank-biserial correlations between $-0.81$ and $-0.96$. This holds under both Pynguin configurations. The reason is visible in the failure counts rather than the scores: Pynguin produced no suite at all for 39 of 60 targets in its default configuration. Disabling the master–worker split, which is the documented workaround for the serialisation crash responsible, reduced that to 35 — a net movement of four that conceals 17 targets gaining a suite and 13 losing one, with 14 of the 17 new suites failing at collection, leaving T4 unchanged at 2/60. We report this because it is the kind of result that disappears when a study reports only the configuration that worked better. Here, neither configuration worked.

On Java the count ordering reverses: both search-based tools detect faults on more targets than the model in every pairing (Section 4.3). The reversal is firmer for EvoSuite, where the paired test is significant under the whole-suite gate ($p = 0.028$), than for Randoop, where no gate reaches significance; we therefore read the Java result as a consistent count advantage for the mature tools rather than a uniformly significant one. Java is where these tools have been developed and tuned for over a decade, and where static types give a search-based generator the information it needs to construct calls — exactly the information the language model was shown to be missing in Section 5.1. Python removes that information, and the ordering flips.

The conclusion is therefore not that language models are better or worse at test generation. It is that the two families fail differently, and that which one leads depends on whether the language gives a search-based tool the types it relies on. A study on Java alone and a study on Python alone would each have produced a confident and opposite headline.

### 5.4 What This Means for Evaluating Test Generators

Three practices follow from what went wrong here rather than from what worked.

*Report the sampling funnel.* Only 26 % of the medium-complexity functions in these ten projects are simultaneously public and unambiguously named. A benchmark that samples on complexity alone therefore spends most of its mass on targets no external generator can address, and every tool evaluated on it is charged for failures that belong to the sampling. The funnel costs nothing to report and bounds how much of a measured failure rate is attributable to the tool at all.

*Rest conclusions on a tier that cannot be satisfied vacuously.* Compilation success is defeated by reflection, which turns names into strings the compiler does not check. A green-test criterion is defeated by a suite containing one assertion-free call to the wrong function — an example that occurs in this corpus and passes both tiers. Neither observation requires an adversary; both arose from ordinary tool output.

*Assume the harness is broken until a failure proves otherwise.* The failure mode that cost the most effort in this study was not a tool performing badly. It was a measurement returning a plausible number after it had stopped measuring: a hard-coded constant, a validity flag counting execution rather than success, reflection turning every method name into a string the compiler never checks, a version mismatch discovering no tests while exiting successfully, a JDK mismatch doing the same, and a green-test gate implemented differently in the two languages under one name. Six such defects surfaced, four of them introduced by us while repairing the other two. None raised an error. The practical implication is that a harness needs a test that fails: if the pipeline below a metric is broken completely and the metric does not change, the metric is not measuring that pipeline. We now apply that check before reporting any rate, and it is the single cheapest safeguard we found.

## 6 Threats to Validity

### 6.1 Construct Validity

*What a validity flag measures.* The central construct threat in this area is that the usual "valid suite" indicator does not measure what its name suggests, and fails silently when it does not. We encountered six distinct instances, four of them in our own instrumentation.

First, a compilation flag inferred from a coverage report measures only whether the class appears in that report. Coverage tools enumerate every class in a project, including classes that were never loaded; such a class yields zero covered branches, which the parser then records as a successfully compiled function with zero coverage. Second, a criterion of "at least one test executed" counts a file in which every test fails. Third, reflection removes the compiler's ability to check names, so a compilation-based rate is uninformative for any suite that uses it. Fourth, mismatched tool versions can make a test launcher discover zero tests while exiting successfully.

The fifth is worth stating in full, because believing it would have inverted a published conclusion. Our Java runner and the generated tests were compiled with the default JDK (class file version 61) and executed under the JDK 11 required by EvoSuite (which reads at most 55). The resulting `UnsupportedClassVersionError` was raised inside a child process, so the harness observed only the absence of a result line and recorded "no tests ran" — for every target. EvoSuite scored 0/60, and a paired test against it returned $p = 0.005$ with a rank-biserial correlation of $-1.000$. Had we reported it, the paper would have claimed a statistically significant advantage for the model over EvoSuite; the true figure is given in Section 4. A well-formed $p$-value made the artefact look more credible, not less.

The sixth concerns the green-test gate itself. Our Python implementation retained the passing tests and discarded the failing ones, as T2 specifies; our Java implementation rejected the entire function if any single test failed. The two languages were measuring different constructs under one name, and the Java side contradicted the definition we had registered in advance. We corrected it and report both settings throughout (Section 4); the correction is discussed under conclusion validity below, because it moves results in our own favour.

None of these produce an error. Each returns a plausible number. We therefore report the four nested tiers of Section 3.3 rather than any single flag, and treat only T4 — fault detection within the target's line range — as evidence that a suite tests its target.

*Mutation operators.* Our operator set is deliberately small and identical across both languages, so that the two are measured with the same instrument. It is weaker than a production mutation framework, so absolute mutation scores are not comparable with studies using PIT or `mutmut`; only within-study comparisons are meaningful. The per-function mutant cap differs between the two language pipelines (20 for Java, 30 for Python), an inconsistency inherited from separate implementations. It binds two Java functions — CJ-004 and CJ-026, both sitting exactly at the 20-mutant ceiling under every Java condition — and no Python function, the largest there being CP-025 at 24 candidate sites against a cap of 30. Since T4 asks only whether at least one mutant is killed, the cap bounds the resolution of the continuous score for those two functions rather than their tier outcome.

### 6.2 Internal Validity

*Sampling.* Our own miner mis-attributed one Java target: `LocalTime.getField` is declared `protected`, but the parser matched a same-named declaration in a nested class and recorded it as public, so it passed the public-only filter. An audit against the real declaration line found this to be the only such case in 60 Java functions; we exclude it from the analysis. We record it here because it is precisely the class-attribution error this study is about, reproduced in our own tooling — knowing about a failure mode does not confer immunity to it.

*Concurrency.* Mutation analysis overwrites source files. Running it directly on a shared source tree corrupts any concurrent measurement reading that tree, silently. We mutate an isolated copy placed ahead of the original on the import path, and verify after each run that the original tree is unmodified.

### 6.3 External Validity

*Source exhaustion in Python.* After all testability criteria, only 64 Python functions in the two projects qualify, of which 34 already appeared in our earlier dataset. A strictly held-out Python replication is therefore limited to roughly 30 functions without adding new repositories. Our Python sample is also drawn from two web-oriented libraries and may not generalise to other domains.

*Complexity distribution.* Although the band is $5 \leq CC \leq 10$, the sample concentrates at the lower end (56 of 120 functions have $CC = 5$). Conclusions about the upper half of the band rest on fewer observations.

*Model and configuration.* A single model at a single temperature was used. We do not claim the findings transfer to other models, and the deterministic setting means we do not estimate run-to-run variance.

### 6.4 Conclusion Validity

*Baseline configuration.* Three limitations constrain how far the baseline comparison can be pushed. EvoSuite was run with a classpath containing only the module's own compiled classes: with the full dependency classpath its bytecode reader aborts in a way that produces no output while reporting success, and we were unable to identify the specific offending artifact. EvoSuite may therefore be weaker here than in a fully configured environment. Seven Java targets in one multi-module project could not be recompiled to Java 11 bytecode and fall outside EvoSuite's coverage. Finally, our Randoop version differs from the one used in our earlier study, so Randoop numbers are compared only within this study.

*Pre-registration and two deviations.* Research questions, hypotheses, tool versions, budgets and the two-branch Pynguin design were committed before any baseline was run.

The first deviation added a condition. On inspection, the enriched prompt we had planned to use turned out to lack the worked exemplar, making it zero-shot rather than one-shot and differing from the baseline in two respects at once. We added a corrected condition that holds the exemplar fixed and varies only the target specification, recorded the change in the pre-registration with its date, and left the original hypothesis unaltered. The uncorrected variant is reported only as a secondary observation.

The second deviation changed the instrument after results had been seen, and it changed it in a direction favourable to us, so we state it plainly. The Java green-test gate described above was corrected to match the registered definition of T2. Six functions under the baseline prompt and eight under the target-specified prompt were affected, against two for EvoSuite and one for Randoop — so the correction helps the model more than it helps its competitors. Three safeguards apply. The pre-correction results were written to the repository and committed *before* the corrected measurement was run, so the timestamps preclude selecting between them afterwards. Both settings are reported side by side for every Java comparison. And we state no conclusion that does not hold under both; where the two disagree, we report the comparison as inconclusive. The Python measurements were not re-run, having matched the registered definition from the start.

## 7 Conclusion

We set out to explain why a language model asked to generate unit tests for specific functions fails as often as it does, and found that a large part of the answer lies outside the model. Of 1,848 functions in the medium-complexity band across ten projects, only 26 % are simultaneously public and unambiguously named — the rest cannot be addressed by an external test at all, and a generator evaluated on them is charged for failures that belong to the benchmark. Rebuilding the sample under criteria fixed in advance, and measuring every suite at four nested tiers, changes both the magnitude and the shape of the result.

The central finding is a split that a single metric would have concealed. Supplying the model with the target's fully qualified name, signature, declaring class and a verified constructor call raised the proportion of Python suites that reach the target from 29/60 to 42/60 ($p = 0.003$), while the proportion that detects an injected fault rose from 16/60 to 22/60 and was not significant ($p = 0.33$). The intervention improves arrival, not discrimination. Reported through branch coverage alone it would have looked like an unambiguous success.

The same asymmetry explains why the intervention helps in one language and not the other. The target block supplies identity, and Python resolves identity at run time, so 14 of 17 import failures disappear. Java resolves identity and types, and the compile-failure count does not move at all — 38 before, 38 after, 35 of them the same functions. What remains in both languages is the construction of a receiver in a state that makes the behaviour observable, and neither prompt condition touches it, even when the constructor call handed to the model has been executed and verified.

Against established generators the result does not resolve in one direction. On Python the model beats Pynguin decisively under both of Pynguin's configurations ($p \leq 0.003$ in every pairing); on Java both mature search-based tools detect faults on more targets than the model in every pairing, though the paired test reaches significance only for EvoSuite under the stricter gate. A study conducted in either language alone would have produced a confident headline pointing the opposite way from the other.

Finally, and least expectedly, the study's most transferable output may be a catalogue of ways a measurement can stop working without saying so. Six such defects surfaced — a hard-coded validity constant, a flag counting execution rather than success, reflection turning method names into strings the compiler never checks, two tool-version mismatches that discovered no tests while exiting successfully, and a green-test gate implemented differently in the two languages under one name. Four of the six were introduced by us while repairing the other two, and one of them, had we trusted it, would have supported a statistically significant claim in precisely the wrong direction. None produced an error message. We therefore report the sampling funnel in full, both configurations of every tool we ran in more than one, and both settings of the gate we corrected — and we advance no conclusion that does not hold under both.

**Future Work**

Three directions follow directly. The state barrier is the one that neither better prompting nor a verified constructor removed; whether supplying an executed *state* — a receiver already driven into a configuration where the target's branches are reachable — removes it is an open and answerable question. Second, the Java compile failures should be classified by the type errors that cause them, which would establish whether the typing barrier is genuinely distinct from the addressing barrier or merely a harder instance of it. Third, the funnel numbers reported here come from ten projects in two languages; whether roughly three quarters of medium-complexity functions being unaddressable from outside is a property of these projects or of software more generally is not something a sample this size can settle.

## References


1. Arcuri, A., Briand, L.: A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. Software Testing, Verification and Reliability **24**(3) (2014)
2. Chen, M., Tworek, J., Jun, H., Yuan, Q., et al.: Evaluating large language models trained on code. arXiv preprint arXiv:2107.03374 (2021)
3. Fraser, G., Arcuri, A.: EvoSuite: Automatic test suite generation for object-oriented software. In: Proceedings of the 19th ACM SIGSOFT Symposium and the 13th European Conference on Foundations of Software Engineering (ESEC/FSE) (2011)
4. Fraser, G., Arcuri, A.: A large-scale evaluation of automated unit test generation using EvoSuite. ACM Transactions on Software Engineering and Methodology (TOSEM) **24**(2) (2014)
5. Inozemtseva, L., Holmes, R.: Coverage is not strongly correlated with test suite effectiveness. In: Proceedings of the 36th International Conference on Software Engineering (ICSE) (2014)
6. Just, R., Jalali, D., Inozemtseva, L., Ernst, M.D., Holmes, R., Fraser, G.: Are mutants a valid substitute for real faults in software testing? In: Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering (FSE) (2014)
7. Kerr, N.L.: HARKing: Hypothesizing after the results are known. Personality and Social Psychology Review **2**(3) (1998)
8. Lukasczyk, S., Fraser, G.: Pynguin: Automated unit test generation for Python. In: Proceedings of the 44th International Conference on Software Engineering: Companion Proceedings (ICSE Companion) (2022)
9. Lukasczyk, S., Kroiß, F., Fraser, G.: An empirical study of automated unit test generation for Python. Empirical Software Engineering **28**(2) (2023)
10. McCabe, T.J.: A complexity measure. IEEE Transactions on Software Engineering **SE-2**(4) (1976)
11. Pacheco, C., Lahiri, S.K., Ernst, M.D., Ball, T.: Feedback-directed random test generation. In: Proceedings of the 29th International Conference on Software Engineering (ICSE) (2007)
12. Papadakis, M., Kintis, M., Zhang, J., Jia, Y., Le Traon, Y., Harman, M.: Mutation testing advances: An analysis and survey. Advances in Computers **112** (2019)

13. Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering **50**(1) (2024)
14. Shamshiri, S., Just, R., Rojas, J.M., Fraser, G., McMinn, P., Arcuri, A.: Do automatically generated unit tests find real faults? an empirical study of effectiveness and challenges. In: Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE) (2015)
15. Siddiq, M.L., Santos, J.C.S., Tanvir, R.H., Ulfat, N., Al Rifat, F., Carvalho Lopes, V.: Using large language models to generate JUnit tests: An empirical study. In: Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE) (2024)
16. Tufano, M., Drain, D., Svyatkovskiy, A., Deng, S.K., Sundaresan, N.: Unit test case generation with transformers and focal context. arXiv preprint arXiv:2009.05617 (2020)
17. Wang, J., Huang, Y., Chen, C., Liu, Z., Wang, S., Wang, Q.: Software testing with large language models: Survey, landscape, and vision. IEEE Transactions on Software Engineering **50**(4) (2024)
18. Yuan, Z., Lou, Y., Liu, M., Ding, S., Wang, K., Chen, Y., Peng, X.: No more manual tests? evaluating and improving ChatGPT for unit test generation. arXiv preprint arXiv:2305.04207 (2023)